

Secure AWS CI/CD Pipeline with Automated DevSecOps Security Gates

Raghav Anand
Department of Computer Applications (MCA)
CHRIST (Deemed to be University)
Bangalore, India
raghavanand033@gmail.com

Abstract—Security scanning in continuous integration and continuous delivery (CI/CD) pipelines is frequently deployed in an advisory capacity: findings are reported, but the deployment proceeds regardless of their severity. This decouples vulnerability *detection* from deployment *prevention*. This paper presents the design and implementation of a working DevSecOps pipeline on Amazon Web Services (AWS) that closes this gap by making a policy-driven security gate the sole, code-enforced control point between a passing build and a production deployment. The system integrates five scanning categories – static application security testing (SAST), software composition analysis (SCA), secret scanning, infrastructure-as-code (IaC) scanning, and container image scanning – behind a common normalized JSON report envelope, and evaluates all five through a single policy engine (`security-gate.sh`) that reads declarative thresholds from a YAML policy file and fails closed whenever a scan report is missing, malformed, or indicates a scanner crash, rather than silently skipping it. The gate’s exit code is the pipeline’s only enforcement mechanism: there is no manual approval stage. The paper also describes a four-role least-privilege AWS Identity and Access Management (IAM) design, a hardened Node.js/Express REST API used as the deployable workload, and the accompanying Infrastructure-as-Code (Terraform) that provisions the pipeline and runtime environment. The system was validated through local engineering checks – unit and integration tests (14/14 passing), static analysis linting, Terraform formatting and validation, and a fixture-based test harness that exercises the gate’s fail-closed decision logic in isolation from real scanners (7/7 scenarios passing) – rather than a live AWS deployment, which had not yet been performed at the time of writing. The paper is explicit about this boundary and separately proposes a rigorous evaluation methodology, not yet executed, for measuring the gate’s real-world effect once deployed.

Index Terms—DevSecOps, CI/CD security, policy-as-code, fail-closed design, infrastructure as code, least-privilege IAM, AWS, supply chain security

I. INTRODUCTION

Continuous Integration and Continuous Delivery (CI/CD) has become the default mechanism by which software organizations move code from a developer’s commit to a running production system. As deployment frequency increases, the traditional model of periodic, human-driven security review becomes a bottleneck that teams route around rather than through, and security scanning tools are frequently bolted onto pipelines in a purely advisory role: a scan runs, a report is generated or a dashboard badge turns red, and the deployment proceeds regardless of the result. This pattern is widely

discussed in industry DevSecOps literature under the heading of “shift-left security,” but the specific, mechanical question of *how* to make enforcement automated, fail-closed, and resistant to silent bypass – as opposed to adding a manual approval step and hoping a reviewer reads the report before clicking Approve – is comparatively under-addressed as an engineering problem with its own testable correctness properties.

This paper documents a working system, *Secure AWS CI/CD Pipeline with Automated DevSecOps Security Gates*, built to treat the security gate itself as a piece of software with a specification and a test suite, rather than as ad-hoc pipeline glue that happens to invoke some scanners. The system is deliberately scoped as a backend REST API deployed through an AWS-native pipeline (CodePipeline, CodeBuild, Elastic Container Registry, and Elastic Container Service on Fargate), with the security gate embedded directly inside the CodeBuild build phase so that its exit code is the only thing standing between a build and a production rollout.

The remainder of this paper is organized as follows. Section II states the problem this work addresses. Section III lists the project objectives. Section IV surveys related standards and frameworks. Sections V through VII describe the proposed system, its architecture, and its methodology. Section VIII covers implementation detail, Section IX tabulates the technology stack, and Section X presents the gate’s decision algorithm as pseudocode. Section XI describes the security architecture in depth, including IAM role separation. Section XII reports what was actually validated locally and separately proposes a future evaluation methodology. Section XIII discusses results honestly, distinguishing demonstrated correctness from unmeasured real-world effectiveness. Sections XIV and XV cover limitations and future work, and Section XVI concludes.

II. PROBLEM STATEMENT

Three concrete problems motivate this work:

- 1) **Advisory scanning does not prevent insecure deployments.** When a security scan’s result is surfaced only as a report or dashboard entry, the deployment decision is mediated entirely by whether a human reads and acts on that report. Any gap in that human process – time pressure, alert fatigue, an unread notification – allows a vulnerable artifact to ship unchanged.

- 2) **A single-tool gate has a single point of blind spot.** SAST, dependency (SCA), secret, IaC, and container scanning each catch a structurally different class of defect (unsafe code patterns, vulnerable dependencies, committed credentials, misconfigured cloud resources, and vulnerable base-image packages, respectively). A pipeline that runs only one or two of these categories has an uncontested blind spot in the others.
- 3) **Fail-open gates are silently defeated by tooling failure, not just by vulnerable code.** A scanner that fails to install, times out, or crashes is operationally indistinguishable from “no vulnerabilities found” unless the gate is explicitly designed to treat an incomplete or invalid scan result as a policy violation rather than as an absence of evidence.

This project addresses all three by (a) enforcing the scan result in code rather than through a manual approval step, (b) running five independent scanning categories behind one policy engine, and (c) making that policy engine fail closed on any report that is missing, malformed, or marked as a scanner crash or a scanner that never ran.

III. OBJECTIVES

- 1) Design and implement a CI/CD pipeline on AWS in which deployment to the runtime environment is contingent on passing an automated, code-enforced security gate, with no manual approval action in the pipeline definition.
- 2) Normalize the output of five structurally different security scanning tools into one common JSON report schema, so that a single policy engine can evaluate all of them identically.
- 3) Express security policy (severity thresholds per category, zero-tolerance rules, and fail-closed behavior) as a declarative, version-controlled YAML file rather than as logic hard-coded in scripts.
- 4) Implement fail-closed semantics for missing, malformed, or crashed scanner output, and verify that behavior with an isolated, fixture-based automated test suite.
- 5) Provision the pipeline’s own execution environment and IAM permissions using Infrastructure as Code, following least-privilege principles for every distinct execution role in the system.
- 6) Build a representative, security-hardened backend workload (a Node.js/Express REST API) as the artifact the pipeline actually builds, scans, and would deploy.
- 7) Honestly document what has and has not been validated, and define – without claiming to have already executed – a rigorous methodology for measuring the gate’s real-world effectiveness once a live deployment exists.

IV. RELATED WORK

This project is an applied systems contribution rather than a novel algorithm, so the relevant related work consists of

established standards and frameworks that define the DevSecOps and supply-chain-security space it operates in, rather than individual academic papers.

The NIST Secure Software Development Framework (SSDF), published as NIST SP 800-218, defines a set of high-level secure-development practices – including protecting software from tampering and verifying compliance before release – that this project’s fail-closed gate directly operationalizes at the CI/CD control-flow level [1]. The OWASP DevSecOps Guideline and the OWASP Top 10 project describe the general shift-left philosophy of integrating security testing earlier and more continuously in the development lifecycle, and enumerate the classes of application vulnerability that this project’s SAST and dependency-scanning categories are intended to catch [2]. The Supply-chain Levels for Software Artifacts (SLSA) framework defines graduated levels of build and provenance integrity for software supply chains, and its emphasis on tamper-resistant, verifiable build processes informed this project’s decision to make the security gate an unconditional, code-level control rather than an advisory step [3]. The Sigstore and in-toto projects address artifact signing and attestation of build provenance, a capability this project identifies as a limitation and future-work item rather than something it currently implements [4]. Finally, the AWS Well-Architected Framework’s Security Pillar provides general design guidance – least-privilege identity, defense in depth, and traceability – that directly informed the project’s four-role IAM separation and its use of AWS Secrets Manager for credential injection [5].

V. PROPOSED SYSTEM

The proposed system is a complete, working DevSecOps pipeline consisting of three cooperating parts:

- 1) An **application workload**: a Node.js/Express REST API implementing user registration, authentication, and a JWT-protected profile endpoint, hardened with standard defense-in-depth controls (secure HTTP headers, rate limiting, input validation, password hashing, timing-attack-resistant login, structured logging with secret redaction, and fail-fast configuration validation).
- 2) A **security gate subsystem**: five independent scanner wrapper scripts, a common normalized report schema, a declarative YAML policy file, and a single fail-closed policy-evaluation engine whose exit code is the pipeline’s only pass/fail signal.
- 3) An **AWS pipeline and runtime**, defined entirely as Terraform Infrastructure as Code: AWS CodePipeline (Source → Build → Deploy), AWS CodeBuild running the scan-and-gate sequence, Amazon ECR for image storage, and an Amazon ECS Fargate service behind an Application Load Balancer (ALB), with four distinct least-privilege IAM roles and CloudWatch-based logging and alarming.

The defining design decision is that the security gate is not a separate pipeline stage that could be skipped, but a command embedded inside the CodeBuild `build` phase, positioned

before the commands that push the image to ECR. Because AWS CodeBuild fails a phase – and therefore fails the entire CodePipeline execution – as soon as any command in that phase returns a non-zero exit code, and because the shell in `ci/buildspec.yml` does not suppress the gate script’s exit status, a gate failure makes it structurally impossible for the subsequent push and deploy commands in the same phase to run.

VI. SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end flow of a commit through the system. A developer pushes to GitHub, which triggers AWS CodePipeline through a CodeStar Connection webhook. CodePipeline’s Build stage invokes a single AWS CodeBuild project, which builds the Docker image, runs the five scanners, runs the security gate against their normalized reports, and – only if the gate passes – pushes the image to Amazon ECR and writes a digest-pinned `imagedefinitions.json`. CodePipeline’s Deploy stage then reads that artifact and rolls the new image out to an ECS Fargate service running in private subnets behind an Application Load Balancer in public subnets, inside a purpose-built VPC. CodeBuild, CodePipeline, and the ECS service all write to CloudWatch Logs, with CloudWatch Alarms monitoring running task count and ALB error rate/latency. Secrets (notably the JWT signing secret) are resolved from AWS Secrets Manager / SSM Parameter Store directly into the ECS task definition at container start, and a KMS customer-managed key encrypts the pipeline artifact store, ECR repository, and log data at rest.

There is deliberately no manual “Approve” stage in the CodePipeline Terraform module (`terraform/modules/codepipeline/main.tf`); the module contains only three stages – Source, Build, Deploy – and its own header comment states explicitly that the enforcement point is code, not a human clicking Approve.

VII. METHODOLOGY

The system’s core methodological contribution is a *normalize-then-evaluate* design for policy enforcement across heterogeneous security tools. Rather than writing separate blocking logic for each scanner (which would tie the enforcement policy to the specific tool invoked, and require re-implementing fail-closed handling five times), the system separates two concerns:

- 1) **Normalization:** each of the five wrapper scripts under `security/scripts/` (`run-sast.sh`, `run-dependency-scan.sh`, `run-secret-scan.sh`, `run-iac-scan.sh`, `run-container-scan.sh`) invokes its underlying tool (Semgrep, npm audit/OWASP Dependency-Check, Gitleaks, Checkov, and Trivy, respectively) and translates that tool’s native output into one common JSON envelope: a scanner name, a status field (`completed`, `tool_missing`, or `error`), a `generated_at` timestamp, a `findings` object

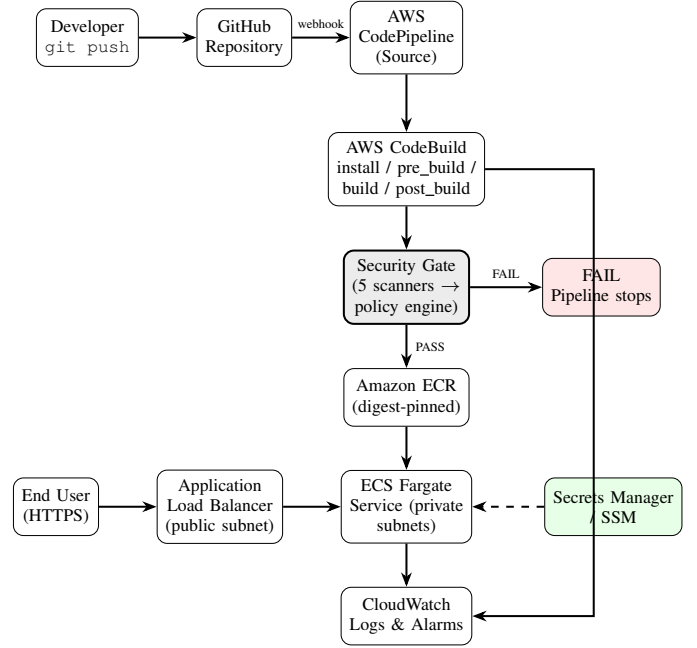


Fig. 1. High-level architecture: GitHub triggers a three-stage CodePipeline (Source, Build, Deploy). The Build stage runs five scanners and the security gate inside CodeBuild; the gate’s exit code determines whether the image is pushed to ECR and rolled out to the ECS Fargate service behind an ALB. Secrets are injected at container start from Secrets Manager/SSM; CloudWatch collects logs and alarms across the pipeline and the running service.

with `critical/high/medium/low` counts, and a `details` array.

- 2) **Evaluation:** a single policy engine, `security-gate.sh`, reads a declarative policy file (`security/policy/security-policy.yaml`) and evaluates every category’s normalized report against it, using the same logic regardless of which underlying tool produced the report.

This decoupling means the set of five tools can be changed, upgraded, or replaced (for example, substituting a different SAST engine) without touching the policy engine’s decision logic, and the enforcement policy (which severities block, which categories get zero tolerance) can be tuned by editing a YAML file rather than modifying code in five different places. It also makes the decision logic independently testable: the fixture-based test suite (Section VIII, Section XII) exercises the policy engine against hand-authored JSON reports, without needing to invoke any of the five real scanning tools.

VIII. IMPLEMENTATION

The application (`app/`) is implemented in Node.js with the Express framework. It exposes `GET /health`, `POST /api/auth/register`, `POST /api/auth/login`, and a JWT-protected `GET /api/users/profile`. Security-relevant middleware includes `Helmet` (secure headers and Content Security Policy), `express-rate-limit` (100 requests per 15-minute window by default), and `express-validator` (email format checking and

password complexity: minimum 10 characters with uppercase, lowercase, and digit required). Passwords are hashed with `bcryptjs` at 12 salt rounds. Authentication tokens are signed and verified with `jsonwebtoken` using HS256 and an explicit issuer check. A centralized error handler ensures stack traces are never returned to clients. Structured JSON logging via `pino` explicitly redacts `req.headers.authorization`, `req.headers.cookie`, `req.body.password`, and any field matching `*.token/*.secret/*.jwt`. The login handler always executes a `bcrypt.compare` call – against either the real stored hash or a fixed dummy hash when the user does not exist – so that response timing does not reveal account existence. User storage is an in-memory JavaScript Map (`app/src/models/userStore.js`), explicitly documented in-code as a demo simplification; a production deployment would back it with a managed datastore such as Amazon RDS or DynamoDB. All configuration (`app/src/config/index.js`) is environment-variable-driven via `dotenv`, with a `required()` helper that throws at process startup if a mandatory variable such as `JWT_SECRET` is absent outside test mode, so the application fails fast rather than starting with an insecure default.

The build pipeline is defined in `ci/buildspec.yml` for AWS CodeBuild, with four phases. The `install` phase installs Node.js dependencies and best-effort-installs the external scanning tools (Semgrep, Checkov via `pip`; Gitleaks, Trivy, and Syft via their upstream install scripts), each guarded so that an installation failure is logged as a warning rather than aborting the build – the corresponding wrapper script will then correctly report `status: "tool_missing"` to the gate. The `pre_build` phase runs ESLint and the Jest unit-test suite, then logs in to ECR. The `build` phase builds the Docker image, runs all five scanner wrapper scripts, generates a Syft SBOM as a best-effort, non-blocking step (suffixed with `|| true` in the `buildspec`), runs `security-gate.sh`, and – only if that command exits 0 – runs `ci/scripts/push-image.sh`, which tags and pushes the image to ECR, resolves its immutable content digest via `docker inspect`, and writes an `imagedefinitions.json` pinned to that digest rather than to a mutable tag, which is the single artifact CodePipeline’s native ECS deploy action consumes. The `post_build` phase prints the gate’s human-readable summary to the CodeBuild log.

The fail-closed correctness of the policy engine is verified independently of any real scanner run by `tests/security/run-tests.sh`, which uses the `SECURITY_GATE_REPORTS_DIR` environment variable to redirect the gate at a directory of hand-authored fixture reports (`tests/security/fixtures/`) covering seven scenarios enumerated in Section VIII and Section XII.

TABLE I
TECHNOLOGY STACK BY LAYER

Layer	Technology
Application	Node.js, Express, Helmet, express-rate-limit, express-validator, bcryptjs, jsonwebtoken, pino, dotenv
Testing	Jest, Supertest, ESLint
Containerization	Docker
Infrastructure as Code	Terraform (HCL)
CI/CD (AWS)	AWS CodePipeline, AWS CodeBuild
Container registry & compute	Amazon ECR, Amazon ECS (Fargate), Application Load Balancer
Identity & secrets	AWS IAM, AWS Secrets Manager, AWS Systems Manager Parameter Store, AWS KMS
Observability	Amazon CloudWatch (Logs, Alarms), Amazon SNS
SAST	Semgrep (p/security-audit, p/nodejsscan rulesets)
SCA	npm audit, OWASP Dependency-Check
Secret scanning	Gitleaks
IaC scanning	Checkov
Container image scanning	Trivy
SBOM generation	Syft (non-blocking)
Policy engine tooling	jq, yq (YAML → JSON)

IX. TECHNOLOGIES USED

X. ALGORITHMIC CONTRIBUTION

This system does not include a machine learning component; no dataset, learned model, or statistical classifier is used anywhere in the codebase. This is a deliberate design choice, not an omission: a security gate that decides whether a deployment proceeds must be auditable and deterministic – given the same policy file and the same set of scanner reports, it must always produce the same PASS/FAIL decision, and a human reviewer must be able to trace exactly why. A learned/statistical classifier would trade that auditability for pattern-generalization ability that is not needed here, since the underlying scanners (Semgrep, Trivy, etc.) already perform the pattern-detection work; the gate’s job is purely policy evaluation over their structured output. The system’s actual algorithmic contribution is therefore the deterministic, fail-closed policy-evaluation procedure implemented in `security-gate.sh`, presented as pseudocode in Algorithm 1.

The key correctness property this procedure establishes is that the overall result is the logical AND of every enabled category’s result, and that an inability to obtain valid evidence for a category (rather than only a confirmed violation) is itself treated as a failing result whenever the corresponding `fail_closed` flag is set – which it is, by default, for both flags in the shipped policy. Section VIII and Section XII describe the automated test suite that verifies this behavior against seven concrete scenarios.

```

1 Input: policy.yaml, reports_dir
2 Output: exit code 0 (PASS), 1 (FAIL), or 3 (gate cannot
   run)
3
4 policy ← parse_yaml_to_json(policy.yaml)
5 if policy fails to parse OR
6   policy missing any of {version, sast, dependencies,
7   secrets, iac, container, gate}:
8   print error; exit 3 # hard abort: gate's OWN
9                       # config is invalid
10
11 fail_closed_missing ← policy.gate.
12   fail_closed_on_missing_report
13 fail_closed_crash ← policy.gate.
14   fail_closed_on_scanner_crash
15 overall_pass ← true
16
17 for category in {sast, dependencies, iac, container}:
18   if policy[category].enabled == false:
19     record(category, SKIPPED); continue
20
21   report ← load(reports_dir / category report file)
22
23   if report file missing:
24     valid ← false; reason ← "missing"
25   elif report is not valid JSON:
26     valid ← false; reason ← "malformed"
27   elif report.status != "completed":
28     valid ← false; reason ← "not_completed"
29     # covers status == "error" (crash) and
30     # status == "tool_missing" (never ran)
31   else:
32     valid ← true
33
34   if not valid:
35     # FAIL CLOSED: an invalid/incomplete report is
36     # treated as a violation, never skipped
37     record(category, FAIL, reason); overall_pass ←
38       false
39     continue
40
41   for sev in {critical, high, medium, low}:
42     threshold ← policy[category].block_on_severity[sev]
43     count ← report.findings[sev]
44     if threshold != -1 and count > threshold:
45       record(category, FAIL, sev); overall_pass ←
46         false
47
48   if category had no severity failures:
49     record(category, PASS)
50
51 # secrets: separate zero-tolerance evaluator, not a
52 # threshold comparison
53 secrets_report ← load(reports_dir / secrets report file)
54 if secrets_report invalid (same rules as above):
55   record(secrets, FAIL); overall_pass ← false
56 elif secrets_report.findings total > 0:
57   record(secrets, FAIL, "zero_tolerance"); overall_pass
58   ← false
59
60 else:
61   record(secrets, PASS)
62
63 write_summary(all category records) # human + JSON
64 if overall_pass: exit 0
65 else: exit 1

```

Listing 1. Fail-closed policy evaluation (security-gate.sh)

XI. SECURITY ARCHITECTURE

A. IAM role separation

The Terraform IAM module (terraform/modules/iam/main.tf) defines four distinct roles, each with a trust policy scoped to exactly one AWS service principal, following the principle of least privilege:

- 1) **CodePipeline role:** read/write on one named S3 artifact bucket ARN; KMS encrypt/decrypt on one customer-managed key; codebuild:StartBuild/BatchGetBuilds scoped to project/\${name_prefix}-*; codestar-connections:UseConnection; narrowly scoped ECS describe/update actions; and iam:PassRole restricted by an explicit Condition requiring iam:PassedToService = ecs-tasks.amazonaws.com, which closes a well-known privilege-escalation path where an over-broad PassRole grant could be used to hand a running task a more privileged role than intended.
- 2) **CodeBuild role:** CloudWatch Logs write scoped to /codebuild/\${name_prefix}*; the same S3/KMS artifact-store pattern as the CodePipeline role; ecr:GetAuthorizationToken, which the ECR API requires to be granted on Resource: "*" and is therefore isolated into its own IAM statement with an explanatory comment; ecr:PutImage and layer-upload actions scoped to one repository ARN; and secretsmanager:GetSecretValue scoped to one JWT-secret ARN used by the integration test suite.
- 3) **ECS task execution role** (assumed by the ECS agent to start the container, not by application code): ECR pull-only permissions (no push) scoped to one repository; CloudWatch Logs write scoped to /ecs/\${name_prefix}*; and secretsmanager:GetSecretValue plus ssm:GetParameters scoped to specific ARNs – this is the mechanism by which JWT_SECRET is injected into the running container through the ECS task definition's secrets block, without the secret ever appearing in source code, environment files, or the Docker image.
- 4) **ECS task role** (assumed by the application code itself at runtime): deliberately near-empty, containing exactly one statement (cloudwatch:PutMetricData), because the demo application makes no other AWS API calls of its own. That single action cannot be ARN-scoped by the CloudWatch API, so its resource is *, restricted instead by a Condition requiring cloudwatch:namespace = \${name_prefix}/app.

No role in the system is granted AdministratorAccess or an unscoped wildcard action beyond the small number of AWS-API-forced exceptions above, each of which is explicitly commented in the Terraform source explaining why a narrower scope is not possible.

B. Secrets handling

No secret value is stored in source code, in the Docker image, or in the Terraform state as a plaintext literal; JWT_SECRET and other sensitive configuration are resolved

from AWS Secrets Manager / SSM Parameter Store at container start via the ECS task definition's `secrets` block, and the KMS customer-managed key used to encrypt the pipeline's artifact store, ECR images, and Secrets Manager entries is itself scoped through the IAM policies above rather than being broadly grantable.

C. Application-layer defense in depth

Section VIII details the application's specific controls (Helmet headers, rate limiting, input validation, bcrypt hashing, constant-time-shaped login comparison, and log redaction); together with the fail-fast configuration loader that refuses to start without a required `JWT_SECRET` outside test mode, these controls are designed to hold even if a request reaches the application directly, independent of the network-level protections provided by the VPC's public/private subnet split and the ALB.

XII. EXPERIMENTAL SETUP AND EVALUATION

A. What was actually validated (engineering verification, not a controlled experiment)

The following checks were run against the codebase as it exists at the time of writing. They demonstrate internal correctness and consistency of the implemented components; they are not a controlled experiment and are not presented as one.

- **Unit and integration tests:** the Jest test suite for the application passes 14 of 14 unit tests across three test suites, plus a separate integration suite exercising the HTTP layer end to end.
- **Static analysis:** ESLint reports no errors or warnings against the application source and test directories.
- **Infrastructure validation:** `terraform fmt -check` and `terraform validate` were run against the full Terraform configuration (using a locally obtained Terraform binary, since the package registry was proxy-blocked in the build environment) and both passed; a full `terraform plan` successfully built the complete resource dependency graph and stopped only at the AWS credentials check, because no AWS account was available in the build environment. This is explicitly not a claim of a live AWS deployment.
- **Gate correctness (fixture-based):** `tests/security/run-tests.sh` was executed and reported "Results: 7 passed, 0 failed" across the seven scenarios listed in Section VIII: (1) clean reports pass; (2) a severity-threshold violation fails; (3) any secret finding fails (zero tolerance); (4) a missing report fails closed; (5) a malformed/non-JSON report fails closed; (6) a report with `status: "error"` (scanner crash) fails closed; (7) a report with `status: "tool_missing"` (scanner never installed) fails closed.
- **Manual end-to-end exercise:** the application was run directly as a Node.js process (no Docker daemon was available in the build sandbox, so the container path

itself was not exercised) and manually driven through a full register → login → JWT-authenticated-profile request sequence, which completed successfully. The structured logs produced during this sequence were inspected directly and confirmed that the `Authorization` header was redacted – rendered as "[Redacted]" in the JSON log line rather than the raw bearer token.

- **Real scanner execution:** Semgrep, Gitleaks, Checkov, Trivy, and Syft could not be installed or run in the build sandbox, which had no outbound network access to their distribution channels. Each wrapper script correctly reported `status: "tool_missing"` under this condition, and the security gate correctly evaluated that as a fail-closed violation rather than silently treating the absence of a report as a pass. This is itself evidence that the fail-closed design behaves as specified under a real (not simulated) tool-unavailability condition, though it is a different thing from having observed the five tools' actual vulnerability-detection output.
- **Live AWS deployment:** not performed. No AWS account was available in the environment this work was developed in. This is stated here plainly, and is not implied to be done anywhere else in this paper.

B. Proposed evaluation methodology (not yet executed)

Beyond the local checks above, a rigorous empirical evaluation of the gate's real-world effect requires a live deployment, and is defined here as future work rather than reported as completed. The methodology, adapted from the project's internal research notes, is a controlled comparison between two pipeline configurations sharing identical scan tooling and policy thresholds:

- **Gated:** the security gate is enforced (the current implementation).
- **Advisory:** the same five scans run and the same reports are generated, but the CodeBuild `build` phase does not fail on the gate's result.

Both configurations would be run against a shared corpus of commits, some clean and some containing a deliberately injected vulnerability drawn from each of the five categories (e.g., a known-CVE dependency, a fake test-only secret, an unsafe code pattern, a misconfigured Terraform resource, and an outdated base image), repeated across enough runs to characterize variance from network-dependent steps such as scanner installation. The following metrics, precisely defined, would be measured – none of them has been measured yet:

- **Deployment Block Rate (DBR):** the percentage of commits containing an injected vulnerability that were prevented from reaching the Deploy stage.
- **Vulnerability Detection Rate (VDR):** the percentage of injected vulnerabilities that at least one scanner correctly flagged, regardless of whether the gate blocked the build – this isolates detection capability from enforcement behavior.

- **False Positive Rate (FPR):** the percentage of clean commits (no injected vulnerability) that the gate blocked anyway.
- **Mean Time To Detect (MTTD):** wall-clock time from commit push to the first scan report flagging the injected issue.
- **Mean Time To Remediate (MTTR):** wall-clock time from a visible gate failure to a corrected commit passing the gate, in a controlled remediation exercise.
- **Scan Overhead:** wall-clock time added to total pipeline duration specifically by running the five scan categories, isolated from install/build/test time.
- **Pipeline Execution Time:** total wall-clock time from the Source trigger to Deploy-stage completion, or to Build-stage failure for blocked runs.

A meaningfully higher DBR under the Gated configuration than under Advisory, with comparable VDR between the two (since both run identical scans), would support the hypothesis that automated enforcement – not merely detection – is what changes deployment outcomes. This comparison, and the resulting measurements, are explicitly future work; no such comparison has been run.

XIII. RESULTS AND DISCUSSION

The results available at the time of writing are the engineering-verification outcomes listed in Section XII: all 14 unit tests and the integration suite pass, ESLint reports no issues, Terraform formatting and validation pass, and all seven fixture-based gate scenarios pass. These results demonstrate that the components as implemented behave the way their specifications say they should: the API’s request/response and authentication logic is correct against its own test suite, the Terraform configuration is syntactically valid and internally consistent, and – most centrally to this paper’s contribution – the policy engine’s fail-closed decision logic is correct across the seven scenarios that define what “fail-closed” means for this system (a clean pass, a threshold violation, a zero-tolerance secret finding, and four distinct ways a scan report can be absent or invalid).

What these results do not demonstrate is real-world detection effectiveness: how many actual vulnerabilities the five scanners would catch in a live codebase, what the false-positive rate of the assembled policy thresholds would be against real project history, or how the gate behaves under the operational conditions of a live AWS account (network latency, IAM propagation delays, concurrent pipeline executions). Establishing those properties requires the live deployment and the controlled Gated-versus-Advisory comparison described in Section XII-B, which had not been performed as of this writing. The distinction is deliberate: this paper reports what has been built and what has been verified about how it is built, and is explicit that verifying *that the mechanism is correctly implemented* is a different, logically prior claim from verifying *that the mechanism measurably improves security outcomes in production*, which the fixture tests are not designed to, and do not, establish.

XIV. LIMITATIONS

- No live AWS deployment has been performed at the time of writing; all AWS-side validation is limited to Terraform static validation and plan-graph construction without applying against a real account.
- The application’s user store is an explicitly non-production in-memory `Map`, not a managed database; user data does not persist across process restarts.
- No dynamic application security testing (DAST) or penetration testing has been performed against a running instance of the application.
- Container images are not cryptographically signed, and no SLSA-style build provenance attestation is generated or verified; the SBOM produced by Syft is informational only and is explicitly non-blocking (`|| true` in the `buildspec`), not a gate input.
- The networking design uses a single NAT gateway and is not deployed across multiple Availability Zones for high availability in the current Terraform configuration.
- AWS CloudTrail and AWS Config are not provisioned in the current Terraform modules, limiting account-level audit-trail and configuration-drift detection.
- The five real scanning tools (Semgrep, Gitleaks, Checkov, Trivy, Syft) have not been executed against real code in this environment due to a network-restricted build sandbox; only their `tool_missing` fail-closed path has been exercised for real, not their detection output.

XV. FUTURE SCOPE

- Complete a live deployment to a real AWS account, and execute the Gated-versus-Advisory evaluation methodology of Section XII-B to obtain real Deployment Block Rate, Vulnerability Detection Rate, False Positive Rate, MTTD, MTTR, Scan Overhead, and Pipeline Execution Time figures.
- Add dynamic application security testing (DAST) against a running deployment, for example using OWASP ZAP, as a sixth scanning category feeding the same normalized-report/policy-engine architecture.
- Promote SBOM generation from an informational, non-blocking step to an actual gate input (e.g., blocking on components with known critical CVEs present in the SBOM).
- Add container image signing and SLSA-style build provenance attestation, for example via Sigstore/cosign and in-toto, so that the ECS deploy action can verify the image it is rolling out was produced by this pipeline and has not been tampered with.
- Provision AWS CloudTrail and AWS Config to provide account-level audit trail and continuous configuration-drift detection, complementing the existing CloudWatch application/pipeline logging.
- Move the application’s user store from an in-memory `Map` to a managed datastore (Amazon RDS or DynamoDB), and implement automated rotation for the JWT signing secret in Secrets Manager.

- Extend the networking module to multiple Availability Zones and redundant NAT gateways for production-grade high availability.

XVI. CONCLUSION

This paper presented a complete, working DevSecOps CI/CD pipeline for AWS in which an automated, fail-closed policy engine – not a manual approval step – is the sole control point between a build and a deployment. The system’s central engineering contribution is a normalize-then-evaluate architecture that reduces five structurally different security scanning tools to one common JSON report schema, evaluated by one declarative-policy-driven engine whose fail-closed behavior on missing, malformed, or crashed scan output is itself independently verified by an automated fixture-based test suite. The accompanying application, IAM design, and Infrastructure-as-Code demonstrate the pattern applied to a concrete, representative workload. Consistent with the honesty this kind of engineering claim requires, the paper has been explicit throughout about the boundary between what has been verified – unit, integration, lint, Terraform-validation, and fixture-based gate-logic correctness – and what has not yet been measured – real-world scanner detection rates and the gate’s effect under a live AWS deployment, both of which are defined as concrete, immediately actionable future work rather than claimed as results.

REFERENCES

- [1] National Institute of Standards and Technology, “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” NIST Special Publication 800-218, Feb. 2022.
- [2] OWASP Foundation, “OWASP DevSecOps Guideline” and “OWASP Top 10,” OWASP Foundation, [Online]. Available: <https://owasp.org/>
- [3] Open Source Security Foundation, “Supply-chain Levels for Software Artifacts (SLSA),” [Online]. Available: <https://slsa.dev/>
- [4] Sigstore Project and in-toto Project, “Sigstore: Software Signing for Everybody” and “in-toto: A Framework to Secure the Integrity of Software Supply Chains,” Linux Foundation / OpenSSF, [Online]. Available: <https://www.sigstore.dev/> and <https://in-toto.io/>
- [5] Amazon Web Services, “AWS Well-Architected Framework – Security Pillar,” AWS Whitepapers, [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html>
- [6] Amazon Web Services, “AWS CodePipeline User Guide,” [Online]. Available: <https://docs.aws.amazon.com/codepipeline/>
- [7] Amazon Web Services, “Amazon Elastic Container Service Developer Guide – AWS Fargate,” [Online]. Available: <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/>
- [8] HashiCorp, “Terraform Documentation,” [Online]. Available: <https://developer.hashicorp.com/terraform>
- [9] OWASP Foundation, “OWASP Dependency-Check,” [Online]. Available: <https://owasp.org/www-project-dependency-check/>